

Reviewer Pack — Minimal Generality of Automata over Boolean Algebras Bounds ...

Entry a03f0d3fc815 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 18:59:59 UTC

Minimal Generality of Automata over Boolean Algebras Bounds Circuit Depth

Entry ID: a03f0d3fc815 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 18:59:59 UTC

1. Conjecture

Field A (mathematical branch): Automata Theory (Automata Over Boolean Algebras) **Field B** (complexity object): Boolean Function Complexity: Circuit Complexity

Statement:

For every n -variable boolean function f , the minimal generality g_f of an automaton accepting f is $\Theta(2^n / \log^2(n))$. Specifically, there exist constants $c_1, c_2 > 0$ such that for all $n \geq 40$, $c_1 * 2^n \leq g_f \leq c_2 * 2^n / \log^2(n)$.

Rationale (proposer's reasoning):

Minimal generality of automata provides a measure of the complexity of recognizing a boolean function. By linking it to circuit depth, this conjecture proposes a new tool for analyzing the hardness of boolean functions. Automata over Boolean Algebras are less explored in complexity theory, and their potential in bounding circuit complexity is largely unexplored.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 95926542156ab7ef

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if the Spearman rank correlation coefficient between the measured generality g_f of automata accepting random n -variable boolean functions and the predicted bounds is ≥ 0.8 , for all $n \geq 40$, across at least 30 seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automata theory AND boolean algebras AND circuit depth - boolean function complexity AND automaton generality AND circuit bound - circuit complexity AND automata over boolean algebras AND minimal generality

Top relevant hits considered: - [http://arxiv.org/abs/1306.5577v2] A Note on Elementary Cellular Automata Classification - [http://arxiv.org/abs/1208.2766v1] Topological properties of cellular automata on trees - [http://arxiv.org/abs/1503.03318v2] On the decomposition of stochastic cellular automata - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/2003.09703v1] Variance function of boolean additive convolution - [http://arxiv.org/abs/1208.2764v1] Linear functional classes over cellular automata - [http://arxiv.org/abs/2011.14404v1] State Complexity of the Set of Synchronizing Words for Circular Automata and Automata over Binary Alphabets

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def construct_automaton(f):
        n = int(math.log2(len(f)))
        states = list(range(2**(n+1)))
        transitions = {state: [[None] * 2 for _ in range(2)] for state in states}
        accepting_states = set()

        for i in range(2**n):
            for j in range(2):
                next_state = (i << 1) | j
                if f[i] == j:
                    accepting_states.add(next_state)
                    transitions[state][j, f[j]] = next_state

        return states, transitions, accepting_states
```

```

def calculate_generality(transitions, n):
    total_transitions = sum(sum(1 for _ in row) for row in transitions[0])
    return total_transitions / (2**(n+1))

def calculate_bounds(n):
    c1 = 0.5
    c2 = 1.0
    return c1 * 2**n, c2 * 2**n / math.log(n)**2

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    states, transitions, accepting_states = construct_automaton(f)
    generality = calculate_generality(transitions, n)
    lower_bound, upper_bound = calculate_bounds(n)

    results.append({
        "n": n,
        "generality": generality,
        "lower_bound": lower_bound,
        "upper_bound": upper_bound
    })

return {
    "metric_name": "Generality",
    "metric_value": sum(result["generality"] for result in results) / len(results),
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": all(lower_bound <= generality <= upper_bound for result in results),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    all_results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        all_results.append(result)

    mean_value = sum(result["metric_value"] for result in all_results) / len(all_results)
    support_fraction = sum(1 for result in all_results if result["conjecture_holds"]) / len(all_results)

    if all(result["conjecture_holds"] for result in all_results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in all_results):
        first_failing_seed = next(seed for seed, result in zip(seeds, all_results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31579790.py", line 79, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31579790.py", line 53, in run_trial
    states, transitions, accepting_states = construct_automaton(f)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31579790.py", line 35, in construct_automaton
    transitions[state][j, f[j]] = next_state
            ~~~~~
```

NameError: name 'state' is not defined. Did you mean: 'states'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the test code and ensure that it completes without crashing. If the issue is resolved, re-evaluate the results against the pre-registered support condition.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20165
2	propose	ollama_remote	glm4:latest	0	0	18546
3	propose	ollama_remote	glm4:latest	0	0	15319
4	preregistration	ollama_remote	glm4:latest	0	0	14715
5	novelty	ollama_remote	glm4:latest	0	0	21279
6	novelty	ollama_remote	glm4:latest	0	0	10046
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13665
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32985
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11069
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13572
11	judge	ollama_remote	glm4:latest	0	0	9193

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 180553 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/a03f0d3fc815.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a03f0d3fc815.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a03f0d3fc815.tar.gz (if generated)